

In JavaScript, **object methods** are functions stored as properties of an object. They define the behavior of the object.

In JavaScript, **prototype** is a property of **functions (constructors)**. It lets you add properties and methods that are **shared by all objects** created using that constructor.

```
var Car = function(model, color) {  
    this.model = model;  
    this.color = color;  
};  
  
Car.prototype.display = function() {  
    console.log("Model: " + this.model);  
    console.log("Color: " + this.color);  
};  
  
var car1 = new Car("BMW X5", "Black");  
var car2 = new Car("Audi A4", "White");  
  
car1.display();  
car2.display();
```

Property with Prototype

```
var Car = function(model) {  
    this.model = model;  
};  
  
Car.prototype.company = "BMW";  
  
var car1 = new Car("X5");  
var car2 = new Car("M4");  
  
console.log(car1.company);  
console.log(car2.company);
```

```

1 ▾ var Winston = function(nickname, age, x, y) {
2     this.nickname = nickname;
3     this.age = age + "yrs old";
4     this.x = x;
5     this.y = y;
6 };
7
8 var winstonTeen = new Winston("Winsteen", 15, 20, 50);
9 var winstonAdult = new Winston("Mr. Winst-a-lot", 30, 229, 50);
10
11 ▾ var drawWinston = function(winstObject) {
12     fill(255, 0, 0);
13     var img = getImage("creatures/Winston");
14     image(img, winstObject.x, winstObject.y);
15     var txt = winstObject.nickname + ", " + winstObject.age;
16     text(txt, winstObject.x+20, winstObject.y-10);
17 };
18
19 drawWinston(winstonTeen);
20 drawWinston(winstonAdult);

```

```

1 ▾ var Winston = function(nickname, age, x, y) {
2     this.nickname = nickname;
3     this.age = age + "yrs old";
4     this.x = x;
5     this.y = y;
6 };
7
8 // the draw method
9 ▾ Winston.prototype.draw = function() {
10     fill(255, 0, 0);
11     var img = getImage("creatures/Winston");
12     image(img, this.x, this.y);
13     var txt = this.nickname + ", " + this.age;
14     text(txt, this.x+20, this.y-10);
15 };
16
17 var winstonTeen = new Winston("Winsteen", 15, 20, 50);
18 var winstonAdult = new Winston("Mr. Winst-a-lot", 30, 229, 50);
19
20 winstonTeen.draw();
21 winstonAdult.draw();|

```

You can add **multiple methods and properties** to a constructor's prototype. All objects created from that constructor share them.

```
var Car = function(model, color) {  
    this.model = model;  
    this.color = color;  
};  
  
Car.prototype.start = function() {  
    console.log(this.model + " started");  
};  
  
Car.prototype.stop = function() {  
    console.log(this.model + " stopped");  
};  
  
Car.prototype.display = function() {  
    console.log("Model: " + this.model);  
    console.log("Color: " + this.color);  
};  
  
var car1 = new Car("BMW X5", "Black");  
  
car1.start();  
car1.display();  
car1.stop();
```

Output

```
BMW X5 started  
Model: BMW X5  
Color: Black  
BMW X5 stopped
```

Multiple Properties

```
var Student = function(name) {  
    this.name = name;  
};
```

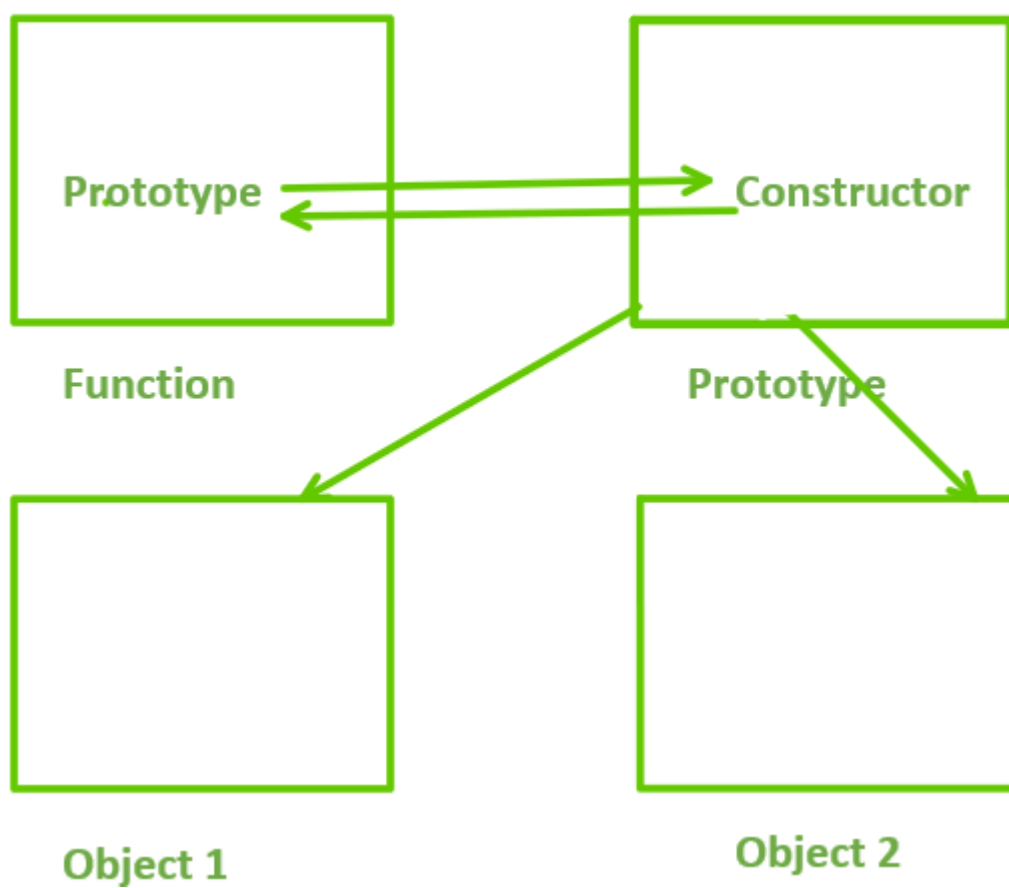
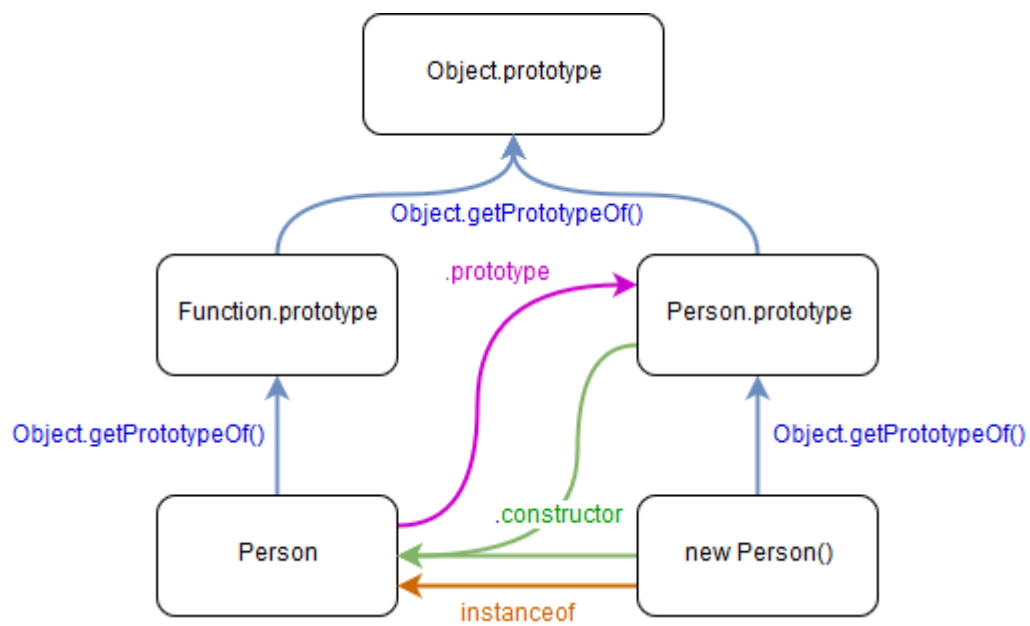
```
Student.prototype.college = "GSFC University";  
Student.prototype.country = "India";
```

```
var s1 = new Student("Ved");
```

```
console.log(s1.name);  
console.log(s1.college);  
console.log(s1.country);
```

```
1 ▾ var Winston = function(nickname, age, x, y) {  
2     this.nickname = nickname;  
3     this.age = age + "yrs old";  
4     this.x = x;  
5     this.y = y;  
6 };  
7  
8 // the draw method  
9 ▾ Winston.prototype.draw = function() {  
10     fill(255, 0, 0);  
11     var img = getImage("creatures/Winston");  
12     image(img, this.x, this.y);  
13     var txt = this.nickname + ", " + this.age;  
14     text(txt, this.x+20, this.y-10);  
15 };  
16  
17 ▾ Winston.prototype.talk = function() {  
18     text("I'm Winston!", this.x+20, this.y+150);  
19 };  
20  
21 var winstonTeen = new Winston("Winsteen", 15, 20, 50);  
22 var winstonAdult = new Winston("Mr. Winst-a-lot", 30, 229, 50);  
23
```

```
winstonTeen.draw();  
winstonTeen.talk();  
winstonAdult.draw();  
winstonAdult.talk();
```



```
var o =  
Object.create(null);
```

null

```
var o =  
{id:42, name: 'John'};
```

Object.
prototype

null

```
var a = ['John'];
```

Array.
prototype

Object.
prototype

null

```
function Entity(id, name) {  
  this.id = id;  
  this.name = name;  
}
```

Entity.
prototype

Object.
prototype

null